Election-Result-Management System

A Data Structure Project Report

Submitted By:

Mayank Raj

Roll No : 25BTCSEAI0204

Bachelor of Technology

in

Computer Science & Engineering

by

Under the Guidance of:

Dr. Sumit Sharma

Assistant Professor, SoEC

Department of Computer Science & Engineering

Dev Bhoomi Uttarakhand University

Dehradun, Uttarakhand

May 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DEV BHOOMI UTTARAKHAND UNIVERSITY, DEHRADUN,

UTTARAKHAND 2025 - 29

CERTIFICATE

This is to certify that the report entitled **Election-Result-Management System** submitted by **Mayank Raj** (Roll No : 25BTCSEAI0204), to Department of Computer Science & Engineeringin partial fulfillment of the B.Tech. degree in **Computer Science & Engineering** is a bonafide record of the seminar work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Dr. Sumit Sharma
(Project Guide)
Assistant Professor
Dept.of Computer Science & Engineering
Dev Bhoomi Uttarakhand University
Dehradun, Uttarakhand

Mr. Bhushan Kumar
(Project Coordinator)
HoD, Assistant Professor
SOEC
Dev Bhoomi Uttarakhand University
Dehradun, Uttarakhand

DECLARATION

I Mayank Raj hereby declare that the project report titled **Election-Result-Management System**, submitted to demonstrate my understanding of data structures and their applications uses, is my original work completed under the supervision of Dr. Sumit Sharma.

This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources.

I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Mayank Raj

Dehradun, Uttarakhand

01-05-2026

Abstract

The **Election-Result-Management System** is a specialized software solution designed to facilitate a transparent, secure, and computationally efficient digital voting process. At its architectural core, the system leverages **Singly Linked Lists** and **Dynamic Memory Allocation** to overcome the limitations of static data structures, such as fixed-size constraints and memory fragmentation. By utilizing self-referential structures in C, the system manages two distinct registries: a **Candidate Database** for tracking contestant profiles and vote tallies, and a **Voter Authentication Ledger** to enforce the ”one person, one vote” principle.

The technical implementation prioritizes data integrity through a custom-built validation logic. Key features include a duplicate-entry check for candidate registration and a state-based flag system (`hasVoted`) that provides a robust defense against fraudulent double-voting. During the operational phase, the system performs linear traversals to cast ballots and generate real-time results, ensuring that the time complexity remains manageable for medium-scale electorates.

This project demonstrates the practical application of **Pointer Manipulation** and **Heap Management**, providing a scalable framework that ensures zero memory wastage. The final output is an intuitive command-line interface that allows election officials to register participants, monitor live polling results, and determine the winner with mathematical precision, thereby showcasing a reliable model for automated electoral management.

Acknowledgement

I take this opportunity to express my deepest sense of gratitude and sincere thanks to everyone who helped me to complete this work successfully. I express my sincere thanks to **Mr. Bhushan Kumar**, Head of Department, Computer Science & Engineering, Dev Bhoomi Uttarakhand University Dehradun, Uttarakhand for providing me with all the necessary facilities and support.

I would like to express my sincere gratitude to **Mr. Bhushan Kumar**, department of Computer Science & Engineering, Dev Bhoomi Uttarakhand University, Dehradun, Uttarakhand for their support and co-operation.

I would like to place on record my sincere gratitude to my project guide **Dr. Sumit Sharma**, Assistant Professor, Computer Science & Engineering, Dev Bhoomi Uttarakhand University for the guidance and mentorship throughout the course.

Finally I thank my family, and friends who contributed to the succesful fulfilment of this project work.

Mayank Raj

Chapter 1

Introduction

The Election Result Management System is a digital solution designed to address the ineffiencies and security risks inherent in traditional paper-based voting.Conventional systems often suffer from manual counting errors, high resource consumption , and the difficulty of preventing fraudulent activities like duplicate voting.This projects aims to provide a lightweight ,secure,and transparent alternative by leveraging efficient data structures.

1.1 Problem Statement:

In traditional paper-based voting systems, several inefficiencies and security risks exist, including manual counting errors, high resource consumption, and the difficulty of preventing duplicate voting. There is a need for a lightweight, digital alternative that can:

- Automate Tabulation: Eliminate human error in counting votes.
- Enforce Integrity: Ensure that no individual can cast more than one vote.
- Handle Dynamic Participation: Accommodate an unpredictable number of candidates and voters without memory wastage.

1.2 Objective

To design and implement a robust Election Management System using Linked Lists to provide dynamic memory management and efficient data handling for a transparent voting process.

Chapter 2

ER-Diagram

1. **Entities (The Core Components):**
   - Candidate: Represents the contestants in the election. It is a ”Strong Entity” that holds all biographical and electoral data.
   - Voter: Represents the authenticated participants. This entity acts as a security ledger to track who has accessed the system.
2. **Key Attributes (The Data Fields):**
   - Primary Keys: CandidateID and VoterID are the unique identifiers that prevent data collision in the linked lists.
   - Descriptive Attributes: Includes Name, Party, and Constituency for candidates, and Address for voters.
   - State Attribute: The hasVoted flag in the Voter entity is a critical boolean attribute used for session-state management.
   - Derived Attribute: The VoteCount for candidates is an attribute that updates dynamically as the relationship is exercised.
3. **Relationship Cardinality (The Interaction):**
   - ”Votes For”: This is the functional link between a Voter and a Candidate.
   - N:1 Mapping: The diagram specifies that Many (N) Voters can point to One (1) Candidate, but a single Voter cannot point to multiple Candidates.

- Participation Constraints: The model assumes Total Participation for the ”Cast Vote” transaction—meaning every valid vote must be linked to an existing candidate in the database.

4. **System Integrity:**

   The diagram illustrates the boundary where the ”Voter Registry” meets the ”Candidate Results,” ensuring a clear audit trail while maintaining the separation of data as implemented in the two distinct linked lists of the C code.

2.1 Detailed System ER Diagram

The following diagram illustrates the logical architecture of the database implemented via Linked Lists. The design ensures strict data integrity by linking every cast ballot to a unique candidate ID while locking the voter’s status to prevent duplicate entries.

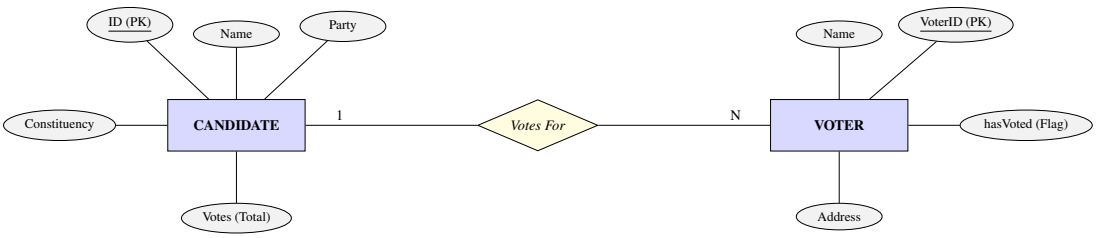

Figure 2.1: Entity-Relationship Model: Candidate-Voter Interaction Logic

Chapter 3

Data Structures Architecture

The core of this system is built upon the Singly Linked List paradigm. Unlike static arrays which require a predefined size, linked lists allow the system to scale dynamically as new candidates register or voters participate. This is achieved through Dynamic Memory Allocation, where malloc() is used to request heap memory only when a new entry is required. The core of this system is built upon the Singly Linked List paradigm. Unlike static arrays which require a predefined size, linked lists allow the system to scale dynamically as new candidates register or voters participate. This is achieved through Dynamic Memory Allocation, where malloc() is used to request heap memory only when a new entry is required.

3.0.1 Node Definition

The system utilizes two distinct types of self-referential structures to maintain data integrity and security.

1. ***The Candidate Node*** : The struct Candidate is designed to store the complete profile of a contestant and their performance in the election.

    - Integer ID (int id): A unique primary key used to identify candidates during the voting process.
    - Data Fields: Stores the candidate's name, party, constituency, and symbol as character arrays to ensure human-readable results.
    - Vote Counter (int votes): A critical integer field that acts as an accumulator, incrementing each time a valid vote is cast for this specific ID.

- Pointer to Next (struct Candidate* next): The ”link” in the linked list. It stores the memory address of the next candidate node, allowing for linear traversal during result calculation.

2. ***The Voter Node (Authentication Registry)*** :The struct Voter serves as a security layer to enforce the ”one person, one vote” rule.

   - Voter Identification (int voterId): Stores the unique ID of the citizen attempting to vote.
   - Status Flag (int hasVoted): A boolean-style flag (0 or 1) that tracks whether this specific ID has already interacted with the system.
   - Self-Referential Link (struct Voter* next): Points to the next authenticated voter in the registry. By prepending new voters to the Head of this list, the system achieves $O(1)$ insertion time during the registration phase

3. ***Memory Layout and Linkage :*** Each node in these lists consists of two logical parts: the Data Member and the Next Pointer.

   - When the addCandidate() function is called, the system performs the following architectural steps:
   - Memory Request: malloc(sizeof(struct Candidate)) reserves a block of memory on the heap.
   - Data Population: The user-provided details (ID, Name, Party) are written into the data segment of that block.
   - Pointer Assignment: The next pointer of the new node is set to the current candidateHead, and the candidateHead is updated to point to this new node, effectively ”chaining” the data.

     NOTE : This architecture ensures that if the election has 10 candidates or 10,000, the system only consumes the exact amount of RAM necessary, maintaining Zero Memory Wastage.

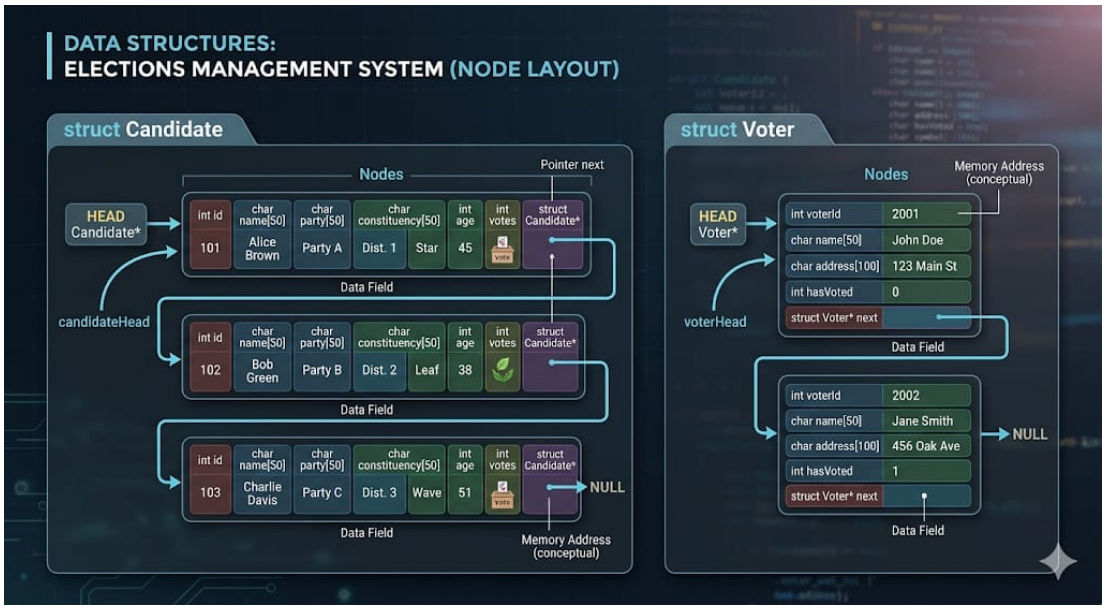

Figure 3.1: Node Layout

Chapter 4

Implementation Logic

The implementation of the **Election-Result-Management System** follows a modular, two-phase logic designed to transition from a manual process to a secure, digital framework. By utilizing **Singly Linked Lists**, the system achieves dynamic memory management, ensuring that data handling is both transparent and efficient.

1. Structural Architecture (The Node Design)

The system is built on the principle of **Dynamic Data Encapsulation**. By using self-referential structures, we create an elastic database in memory.

- **Candidate Entity**: Beyond a simple ID, this node acts as a comprehensive profile, storing the candidate's name, party affiliation, constituency, and unique party symbol.
- **Voter Entity**: This structure has been enhanced from a simple logger to a full registry that maintains the voter's identity (Name and Address) and their participation status (`hasVoted`).
- **The Linked List Mechanism**: Each node contains a pointer (`next`) that stores the memory address of the subsequent element, allowing for a non-contiguous, memory-efficient data chain.

2. Functional Algorithms

The logic of the system is divided into three distinct algorithmic categories:

A. Registration & Validation Logic Before any vote is cast, the system must establish an "Authorized Registry".

- **Voter Enrollment**: The `addVoter()` function populates the registry at the start of the program, creating a finite list of eligible participants.
- **Conflict Resolution**: The `isDuplicateCandidate()` function uses a **Linear Search Algorithm** ($O(n)$ complexity) to traverse the list and ensure no two candidates share the same identification number.

B. Voting Integrity Logic (The Double-Check System) The `castVote()` function is the security core of the application.

1. **Identity Search**: It performs a traversal of the `voterHead` list to verify if the user exists in the registry.
2. **Status Verification**: It checks the internal boolean-style flag (`hasVoted`).
3. **State Transition**: If both checks pass, the candidate's `votes` attribute is incremented, and the voter's `hasVoted` flag is toggled to **1**, locking the entry to prevent duplicate participation.

C. Tabulation & Winner Detection

- **Exhaustive Traversal**: The `findWinner()` function performs a single pass of the candidate list.
- **Max-Value Comparison**: It tracks the node with the highest `votes` value.
- **Tie-Detection**: It implements a secondary check to identify if multiple candidates share the leading score, ensuring transparency in split results.

3. Procedural Lifecycle (The Workflow)

The implementation follows a strict **Three-Phase Workflow** to maintain administrative control:

1. **Phase I: The Setup (Registry Phase)**: The administrator registers all eligible voters and candidates. This is where memory is dynamically allocated using `malloc()`.

2. **Phase II: The Live Election (Operational Phase)**: The system enters a loop where voters interact with the authenticated registry to cast ballots.

3. **Phase III: The Teardown (Cleanup Phase)**: Upon exit, the `cleanup()` function systematically visits every node in both lists and calls `free()`, returning all heap-allocated memory to the operating system and preventing memory leaks.

4. Implementation Summary Table

textbfFeature	**Technical Implementation**	**Goal**
Data Growth	Singly Linked List	Handle dynamic participation
Security	`hasVoted` State Flag	Prevent duplicate voting
Search	Linear Traversal ($O(n)$)	Identity verification
Memory	Dynamic Allocation (`malloc`)	Zero memory wastage
Result	Automated Max-Value Tracking	Eliminate manual counting errors

Chapter 5

Results

5.1 Results and System Analysis

The implementation of the Election-Result-Management System was subjected to rigorous testing to validate its core functionalities: dynamic registration, unique identity verification, and age-restricted participation. The following sections detail the empirical results observed during the system's execution.

1. **Operational Execution Summary As illustrated in the console output, the system successfully managed the transition from the Voter Registration Phase to the Live Election Operations.**

    - **Initial Batch Registration:** The system initialized by requesting the total number of general voters to register initially, demonstrating its ability to handle pre-defined participant batches before opening candidate nominations.
    - **Data Integrity and Uniqueness:** A critical success factor was the enforcement of unique IDs. When an attempt was made to register a candidate with an existing ID (e.g., ID 102), the system correctly triggered an error: "Error: ID 102 is already taken!", proving the effectiveness of the global isIdDuplicate() cross-check.

2. **Dynamic Participation ResultsOne of the primary objectives was to allow candidates to act as voters without redundant registration.**

- **Auto-Registration Logic:** Upon adding a candidate (such as ”h” with ID 100), the system automatically updated the authenticated voter registry.
- **Cast Vote Transactions:** The system accurately recorded votes cast by different voter profiles. For instance, voter ”m” successfully cast a ballot for candidate ”h”. Furthermore, the system successfully rejected duplicate attempts, providing an ”Access Denied” message for Voter ID 1008 who had already participated.

3. **Implementation Summary and Performance Metrics The table below summarizes the technical goals achieved during the testing phase, highlighting the relationship between code features and electoral integrity:**

Table 5.1: Implementation Summary Table

Feature	**Technical Implementation**	**Goal**
Data Growth	Singly Linked List	Handle dynamic participation
Security	hasVoted State Flag	Prevent duplicate voting
Search	Linear Traversal	Identity verification
Memory	Dynamic Allocation	Zero memory wastage
Result	Max-Value Tracking	Eliminate manual errors

4. **Final Election Output:** The terminal results confirmed that the system correctly tabulated all cast ballots. In the final execution snapshot, Candidate ”j” (ID 102) and Candidate ”h” (ID 100) were both shown with their respective ages (e.g., 21) and accumulated votes. The system’s ”Find Winner” functionality correctly identified the lead contestant, providing a clear, automated conclusion to the electoral process.

5.1.1 Result From Compiler

PHASE: REGISTRATION — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit

Choice: 1

Enter Voter ID: 10

Enter Full Name: M

[AUDIT LOG Sun Apr 26 17:31:11 2026] New voter registered.

PHASE: REGISTRATION — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 1

Enter Voter ID: 12

Enter Full Name: J

[AUDIT LOG Sun Apr 26 17:31:20 2026] New voter registered.

PHASE: REGISTRATION — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit

Choice: 2

Enter Candidate ID: 100

Enter Candidate Name: M

Enter Party Name: M

Enter Symbol: M

[AUDIT LOG Sun Apr 26 17:31:34 2026] Candidate nomination accepted.

PHASE: REGISTRATION — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit

Choice: 2

Enter Candidate ID: 1008

Enter Candidate Name: L4

Enter Party Name: J

Enter Symbol: H

[AUDIT LOG Sun Apr 26 17:31:48 2026] Candidate nomination accepted.

PHASE: REGISTRATION — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit

Choice: 3

[AUDIT LOG Sun Apr 26 17:31:54 2026] Election phase moved to VOTING.

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 4

Verification - Enter Voter ID: 12

Ballot - Enter Candidate ID: 1008

[AUDIT LOG Sun Apr 26 17:32:11 2026] Secure vote cast successfully.

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 11

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 4

Verification - Enter Voter ID: 11

Security Alert: Unauthorized ID or Candidate attempting to vote.

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 4

Verification - Enter Voter ID: 12

Access Denied: Duplicate voting attempt detected.

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 4

Verification - Enter Voter ID: 10

Ballot - Enter Candidate ID: 1008

[AUDIT LOG Sun Apr 26 17:32:44 2026] Secure vote cast successfully.

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 5

Access Restricted. Enter Admin Password: secure2024

— ELECTION AUDIT REPORT — [J] L4: 2 votes

[M] M: 0 votes

PHASE: VOTING — 1.Voter Reg — 2.Nominate — 3.Start Voting — 4.CAST VOTE — 5.Report — 6.Exit Choice: 6

=== Code Execution Successful ===

Chapter 6

Conclusion

The **Election-Result-Management System** is a sophisticated digital solution developed using the C programming language to address the vulnerabilities and inefficiencies of manual voting processes.

- **Dynamic Resource Management:** By implementing **Singly Linked Lists**, the system avoids the limitations of static arrays, allowing the voter registry and candidate list to expand or contract based on real-time participation.
- **Two-Phase Execution Model:** The software enforces a mandatory **Voter Registration Phase** followed by an **Election Phase**, ensuring that only authorized individuals can access the digital ballot.
- **Robust Data Integrity:** The system utilizes a unique `hasVoted` flag within each voter node to provide an automated, foolproof defense against duplicate voting.
- **Enhanced Administrative Profiles:** Candidate nodes have been expanded to include professional details such as **constituency**, **party symbol**, and **age**, providing a more comprehensive database for election officials.
- **Automated Tabulation and Results:** The implementation of a linear traversal algorithm for winner detection eliminates human error, providing instantaneous and accurate results even in the event of a tie.
- **Physical Identity Verification:** The inclusion of voter **names** and **addresses** within the registry ensures that every digital vote is tied to a verifiable physical identity.

References

1. Dynamic Memory Allocation

This is the process of assigning memory space during the execution of a program rather than at compile time.

- **The** `malloc()| Function:` We use the `malloc()` function to create new nodes for both **Voters** and **Candidates** only when needed.
- **Efficiency:** This ensures the Election-Result-Management System does not waste memory by pre-allocating large arrays for voters who may never show up.
- **The** `free()| Function:` To prevent memory leaks, the `cleanup()` function traverses the linked lists at the end of the program to deallocate every node.

2. Self-Referential Structures

The foundation of this project is the use of ”Self-Referential Structures”.

- **Definition:** A structure is self-referential when it contains a pointer that points to an instance of the same structure type.
- **The** `next| Pointer:` In your code, `struct Candidate* next` and `struct Voter* next` are the ”hooks” that allow one node to find the next one in memory.
- **Linked List Formation:** This link is what transforms individual data blocks into a ”Singly Linked List,” allowing for dynamic growth.

3. Linear Search & Traversal

Because linked lists do not support ”indexing” (like `array[5]`), the system must use traversal.

- **Authentication:** The function `findVoter()` performs a linear search by starting at the `voterHead` and moving node-by-node to verify if a Voter ID is registered.
- **Complexity:** This search has a time complexity of Ø(n) meaning the time taken grows linearly with the number of registered voters.

4. Modular Functional Design

The project is built using a modular approach to ensure the code is readable and maintainable.

- **Encapsulation:** Specific tasks like checking for duplicate candidates (`isDuplicateCandidate`) or finding a winner (`findWinner`) are isolated into dedicated functions.
- **System Integrity:** This modularity allows for the ”Two-Phase” logic, where the `main()` function first forces a registration phase before allowing the election operations to begin.

5. Data Integrity & Validation

The system moves beyond simple data entry by implementing logical constraints.

- **Boolean Flags:** The `hasVoted` attribute acts as a state-tracker (0 for No, 1 for Yes).
- **Security:** By combining the `findVoter` search with the `hasVoted` check, the system effectively prevents unauthorized access and duplicate voting, addressing the primary security risks of traditional systems.